INTEGRAL DRAGON

MODULE 02 · WORKSHOP 03

Using AI to Inspect and Improve

Targeted, reviewable AI use against a real codebase — inspect, explain, propose, and scaffold, with humans owning the architecture decisions.

AI-ACCELERATED SOFTWARE DEVELOPMENT | APPLIED PRACTICE

What AI is actually good at in existing codebases

Five reliable jobs. High leverage when scope is clear. None of them are "own the architecture decision."

01 · INSPECT

Inspect

Repo summarization, structure scans, cross-file pattern detection.

02 · EXPLAIN

Explain

Module roles, call paths, legacy logic walkthroughs.

03 · PROPOSE

Propose

Ranked findings, small refactor candidates, test ideas.

04 · SCAFFOLD

Scaffold

First-draft docs, test skeletons, migration outlines.

05 · STANDARDIZE

Standardize

Checklists, review templates, repeatable prompts.

Framing line — Use AI to make the hidden visible.

Weak prompt vs. tightened prompt

BEFORE — WEAK

Generic ask, no structure, no criteria

```
Analyze this codebase and improve it.
```

Produces generic, plausible prose. No evidence trail. No ranked findings. Uncertainty hidden.

AFTER — TIGHTENED

Scoped, criteria-driven, inspectable output

```
Act as a senior architect. Evaluate this Python/FastAPI repo
against 9 project-health categories. For each: confirmable
evidence, weaknesses, likely debt, 1–5 rating, confidence
level. Rank the top 5 problems. Distinguish facts from
inferences. Do not hallucinate architecture.
```

Produces structured findings with explicit uncertainty — an artifact a reviewer can check.

Five levers — scope · quality criteria · output structure · uncertainty · validation expectations.

AI for documentation and architecture discovery

One of the safest high-value uses of AI is making hidden knowledge explicit — then reviewing it against the repo.

DOCS · 01

README refresh candidates

Generate draft sections grounded in the repo's actual structure. Flag what the current README claims but the code does not support.

DOCS · 02

Module role summaries

One-paragraph summaries of each top-level module with its likely responsibilities and obvious collaborators.

ARCHITECTURE · 03

C4-style component overview

First-pass container and component map. Treat as a draft needing human edit, not a source of truth.

ARCHITECTURE · 04

Unanswered questions list

Explicit uncertainty — what the repo does not answer — so reviewers know where to look next.

AI for tests and small refactors

AI accelerates confidence-building work when the scope is constrained. Small, scoped, reviewable — always.

- → **Candidate tests for one component** · generate ideas, then filter for real behavior over implementation trivia.
- → **Seam creation** · propose small isolation changes before attempting bigger refactors.
- → **Logging and error-handling improvements** · repetitive cleanup under clear rules you can review line by line.
- → **Small, scoped diffs** · split large proposals before merging anything — oversized diffs are a failure mode, not a feature.

AI failure modes — the red flags

The biggest risk is fluent overreach. Fluency is not evidence. Participants need permission to distrust impressive-looking AI output.

RED FLAG 01

Hallucinated architecture

Confident descriptions of flows, services, or boundaries that don't exist in the repo.

RED FLAG 02

Shallow tests

Assertions that exercise lines without testing the behavior that actually matters.

RED FLAG 03

Unsafe refactors

Changes that silently expand scope beyond the stated problem.

RED FLAG 04

Oversized diffs

PRs too large for a reviewer to reason about in one sitting.

RED FLAG 05

Confident summaries

Smooth prose that hides uncertainty instead of surfacing it.

RED FLAG 06

Unrequested complexity

Generated abstractions, patterns, or frameworks no one asked for.

Framing line — Trust evidence, not fluency.

The review bar should be clearer, not lower

RULE 01

Require explicit scope

Name the files, the behavior, and the acceptance before the AI starts. Scope drift is the most common AI failure.

RULE 02

Require validation steps

Every AI change needs real tests, actual run output, or concrete verification. No evidence, no merge.

RULE 03

Prefer small diffs

Split large AI proposals into reviewable slices. If a human can't hold it in their head, it is not ready.

RULE 04

Check assumptions against the repo

Treat AI output as a draft, not as evidence. Verify every claim of "the repo does X" against the actual code.

This is where AI excitement becomes engineering discipline.

Today's exercises — produce concrete, inspectable artifacts

EXERCISE 03

Documentation Gap Hunt

AI outline vs. real docs. List the missing or misleading items and draft improved README sections.

EXERCISE 04

Architecture Map from the Repo

First-pass component map with responsibilities and explicit uncertainty flags — a one-page artifact.

EXERCISE 05

AI-Assisted Test Improvement

Propose candidate tests for one risky behavior, then filter for real behavior versus implementation trivia.

SUPPORTING ASSET

Participant Prompt Template for Codebase Analysis — a reusable structured prompt against the nine project-health categories.

Debrief question — What AI-generated outputs would you trust after review, and which would you never trust without deeper verification?

• CLOSE OUT

Use AI to inspect, explain, propose, and scaffold.

Leave the architecture decisions to humans who can validate them. Trust evidence, not fluency. Keep every change small, scoped, and reviewable.